Contre-mesure d’une attaque SSH (MITM + ARP Snooping)

Sommaire

Sommaire 1
Informations additionnelles 1
Partie 1 - Mise en place d’une authentification par clés de chiffrement 1
Partie 2 - Utilisation de ssh-agent 6
Partie 3 - Faciliter la vérification de l’identité du serveur SSH avant la première connexion 7

Informations additionnelles

Material/Os utilisé	Kali Linux attaquante / Un routeur OpenBSD / Debian 12 client ssh / Debian 12 Serveur SSH
Procédure réalisée sur machine virtuelle :	Oui

Partie 1 - Mise en place d’une authentification par clés de chiffrement

Nous allons éditer le fichier de configuration du serveur et autoriser le mécanisme de connexion :

```
etusio@srvssh:~$ sudo edit /etc/ssh/sshd_config
```

Il faudra renseigner les valeurs suivantes :

```
PubkeyAuthentication yes \\ A décommenter
AuthorizedKeysFile .ssh/authorized_keys
```

Puis redémarrer le service avec :

```
etusio@srvssh:~$ sudo systemctl restart sshd
```

Nous avons à présent terminé les modifications sur le serveur. Nous allons tester via le client SSH.

Sur la machine cliente, l'utilisateur etui va devoir générer sa clé publique, et sa clé privée afin de pouvoir s'authentifier sur le serveur OpenSSH :

```
etusio@clissh:~ $ ssh-keygen -b 256 -t ecdsa
```

- Appuyer sur Entrée pour accepter la localisation de la clé par défaut.
- ssh-keygen demande ensuite une phrase de chiffrement (ou « passphrase », équivalent d'un mot de passe, mais sous forme de phrase).
  Tapez TPMITMcerta comme phrase de chiffrement et confirmez-là.
- Vous obtenez l'empreinte de votre clé (« key fingerpint ») et une « randomart image ».

Le générateur de clés va en générer deux, une clé publique et une clé privée. Il va placer la clé privée dans un endroit qui, par défaut, est dans :

```
/home/etusio/.ssh/id_ecdsa.
```

```
etusio@clissh:~$ ssh-keygen -b 256 -t ecdsa
Generating public/private ecdsa key pair.
Enter file in which to save the key (/home/etusio/.ssh/id_ecdsa):
/home/etusio/.ssh/id_ecdsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/etusio/.ssh/id_ecdsa.
Your public key has been saved in /home/etusio/.ssh/id_ecdsa.pub.
The key fingerprint is:
SHA256:EUqiJxR0DqCJGjQkTQMkdGN6lIJij3sH5guVLVWocIc etusio@clissh
The key's randomart image is:
```

Q1) Pourquoi l'algorithme ECDSA a été préféré à l'algorithme RSA lors de la génération de la paire de clés ?

L'algorithme ECDSA à été préféré car il permet le même niveau de sécurité mais avec des clés beaucoup plus courtes. Cela offre de meilleures performances donc plus de rapidité.

Q2) À votre avis, pourquoi la mise en place de cette phrase de chiffrement pour accéder à la clé privée est extrêmement importante ?

Cela à été mis en place afin que personne ne puisse y accéder sans connaître la passphrase. Via ceci, on ajoute une sécurité supplémentaire.

→ Que faire de la paire de clés ? Si l'on récapitule, ssh-keygen a généré deux clés.
Une clé privée qui est $HOME/.ssh/id_ecdsa à laquelle vous seul devez avoir accès et une clé publique qui est $HOME/.ssh/id_ecdsa.pub, qui peut être connue par tout le monde.

Q3) Lister le contenu du répertoire $HOME/.ssh/ puis afficher le contenu du fichier id_ecdsa.pub.

```
cd /home/etusio/.ssh
ls
```

```
etusio@clissh:~/.ssh$ ls
id_ecdsa  id_ecdsa.pub  known_hosts  known_hosts.old
etusio@clissh:~/.ssh$
```

```
cat id_ecdsa.pub
```

```
etusio@clissh:~/.ssh$ cat id_ecdsa.pub
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBPl0Meam5okleDdvzAiItwqmwq7bUPf3+yCSkYuxk54DxMOP9vF4bjNrpYPvCdQcOgI9ZWh4oK0duaEvin89Rqk= etusio@clissh
etusio@clissh:~/.ssh$
```

Sur le serveur, vous devez maintenant autoriser explicitement votre compte présent sur la machine cliente à accéder via ssh à celui-ci. Pour ce faire, vous devez ajouter dans le répertoire .ssh de l'utilisateur qui sera choisi pour se connecter (exemple : /home/etusio/.ssh) la clé publique générée précédemment (id_ecdsa.pub) dans un fichier authorized_keys.

La méthode la plus simple est d'utiliser la commande ssh-copy-id qui copiera la clé dans un fichier authorized_keys à l'emplacement défini dans votre chemin(~/.ssh/), même si celui-ci n'existe pas au préalable.

```
etusio@clissh:~$ ssh-copy-id -i ~/.ssh/id_ecdsa.pub
etusio@srvssh.local.sio.fr
```

Il faudra juste entrer le mot de passe.

→ Depuis le poste client, relancez une connexion ssh au serveur, la phrase de chiffrement vous est bien demandée, cela vous permet de vérifier que la connexion avec clé publique est fonctionnelle.

```
etusio@clissh:~$ ssh 192.168.56.10
Enter passphrase for key '/home/etusio/.ssh/id_ecdsa':
```

→ Sur le serveur Debian, désactiver l'authentification par mot de passe pour juste conserver celle par clés dans le fichier de configuration /etc/ssh/sshd_config

```
etusio@srvssh:~$ nano /etc/ssh/sshd_config
```

Il faudra décommenter la ligne suivante :

```
PasswordAuthentication
```

Il faudra la rendre comme ceci

```
PasswordAuthentication no
```

Attention ! Si vous désactivez l'authentification par mot de passe, vous devez vous assurer que l'authentification par clé est opérationnelle sous peine de ne plus du tout pouvoir accéder à la machine distante !

Il faudra ensuite redémarrer le service :

```
sudo service ssh restart
```

Nous allons maintenant vérifier que ssh se préoccupe de la sécurité de vos clés :

Q4. Sur la machine cliente clissh.local.sio.fr, modifier les droits du fichier ~/.ssh/id_ecdsa (droits initiaux : 600 etusio:etusio) qui contient votre clé privée en 644. Se connecter sur le serveur distant srvssh.local.sio.fr qui contient la clé publique. Que se passe-t-il ? Pourquoi ?

Pour modifier les droits :

```
cd /home/etusio/.ssh
sudo chmod 644 id_ecdsa
```

On peut vérifier avec

```
ls -l
```

On se connecter maintenant au serveur SSH

```
etusio@clissh:~$ ssh 192.168.56.10
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0644 for '/home/etusio/.ssh/id_ecdsa' are too open.
It is required that your private key files are NOT accessible by others
This private key will be ignored.
Load key "/home/etusio/.ssh/id_ecdsa": bad permissions
etusio@192.168.56.10: Permission denied (publickey).
```

En effet, nous avons rendu une clé privée, visible aux yeux de tous. Naturellement cela n'est pas recommandé donc le système nous en avertit et bloque la connexion car la clé n'est plus sécurisée.

Si on remet les bons droits, alors la clé privée devrait être de nouveau protégé donc la connexion est possible.

Q5) Rétablir les droits de ~/.ssh/id_ecdsa sur le poste client. Maintenant sur le serveur distant,afficher les droits d'accès appliqués au fichier ~/.ssh/authorized_keys.

```
cd /home/etusio/.ssh
sudo chmod 600 id_ecdsa
```

La connexion est retrouvée.

```
etusio@clissh:~$ ssh 192.168.56.10
Enter passphrase for key '/home/etusio/.ssh/id_ecdsa':
Linux srvssh 4.19.0-8-amd64 #1 SMP Debian 4.19.98-1+deb10u1 (2020-04-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Dec  2 09:11:38 2025 from 192.168.56.11
```

Pour voir les droits sur ~/.ssh/id_ecdsa, on fait :

```
cd /home/etusio/.ssh
ls -l
```

```
etusio@srvssh:~/.ssh$ ls -l
total 4
-rw------- 1 etusio etusio 175 déc.   2 08:54 authorized_keys
```

Il est juste en lecture/écriture.

Q6) Puis, modifier les droits de ~/.ssh/authorized_keys en 666. Se déconnecter puis se reconnecter. Vérifier si un changement est apparu ou non et tenter d'expliquer pourquoi (ne pas oublier de rétablir les droits d'origine ensuite).

Pour modifier les droits on fait :

```
cd /home/etusio/.ssh
sudo chmod 666 authorized_keys
```

```
etusio@srvssh:~/.ssh$ ls -l
total 4
-rw-rw-rw- 1 etusio etusio 175 déc.   2 08:54 authorized_keys
```

La connexion en SSH n'est plus possible car Authorized_keys est en 666, n'importe quel utilisateur peut le modifier → OpenSSH refuse de l'utiliser → connexion par clé impossible.

Afin de rétablir les droits d'origines, on passera par la console car ssh n'est plus possible.

Q7) En analysant la connexion par clés, proposer une hypothèse de fonctionnement de cette nouvelle forme d'authentification. Expliquer ce qui différencie une connexion par mot de passe d'une connexion par clé de chiffrement.

Via cette méthode, la connexion en SSH ne sera possible uniquement si les clés nécessaires sont connues et apprises. Le mot de passe ne sera plus utilisé comme moyen de connexion car contrairement aux clés, il est récupérable si une attaque MITM est en cours. Malgré le fait que les clés de chiffrement soit chiffrés, on peut les voir transiter mais on ne pourra rien en faire.

Partie 2 - Utilisation de ssh-agent

L'agent ssh est particulièrement utile si vous vous connectez très régulièrement à une machine et que vous ne souhaitez pas retaper la phrase de passe liée à votre clé privée à chaque connexion. Mais attention, cela peut ouvrir certaines brèches en matière de sécurité.

→ Sur le client, tapez les commandes suivantes, pour mettre en place le ssh-agent. Par défaut sous Debian, le bureau GNOME intègre un trousseau de clés qui fonctionne en adéquation avec ssh-agent et qui permet d'éviter de saisir à chaque fois des mots de passe.

```
etusio@clissh:~$ exec ssh-agent $SHELL
```

NB : L'utilisation de la variable $SHELL permet d'exécuter l'agent SSh dans l'interpréteur en cours d'utilisation.

Entrer la phrase de passe configurée au préalable. Pendant toute la durée de la connexion, il est possible d'avoir accès à la machine distante sans avoir à taper un mot de passe. Si le processus ssh-agent disparaît, il sera nécessaire de retaper le mot de passe.

```
etusio@clissh:~$ ssh-add
```

```
etusio@clissh:~$ ssh-add
Enter passphrase for /home/etusio/.ssh/id_ecdsa:
Identity added: /home/etusio/.ssh/id_ecdsa (etusio@clissh)
etusio@clissh:~$
```

Q8. Suite à la mise en place de cette authentification par clés de chiffrement, tenter à nouveau de simuler une attaque MITM entre le client et le serveur SSH. Quel résultat obtenez-vous ? Pourquoi ?

On remarque que dès qu'on lance le script et l'empoisonnement ARP et qu'on tente de se connecter, cela échoue.

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Dec  2 08:54:53 2025 from 192.168.56.1
etusio@clissh:~$ ssh 192.168.56.10
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!     @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:sRqG2SHLNSsG/S22CpPdk699/D+InpQscGTVYLNXx8E.
Please contact your system administrator.
Add correct host key in /home/etusio/.ssh/known_hosts to get rid of this mes
.
Offending ECDSA key in /home/etusio/.ssh/known_hosts:3
  remove with:
  ssh-keygen -f "/home/etusio/.ssh/known_hosts" -R "192.168.56.10"
ED25519 host key for 192.168.56.10 has changed and you have requested strict
cking.
Host key verification failed.
```

De plus, si on remarque qu' aucun mot de passe n'a été récupéré.

→ Effacer à nouveau le contenu du fichier /home/etusio/.ssh/known_hosts avant de poursuivre la suite du TP et penser également à arrêter l'attaque si cela n'a pas encore été fait.

```
echo > ~/.ssh/known_hosts
```

Partie 3 - Faciliter la vérification de l'identité du serveur SSH avant la première connexion

OpenSSH adopte par défaut un modèle de sécurité appelé Trust On First Use (TOFU). Lors de la première connexion et à défaut de pouvoir authentifier l'hôte, ssh demande confirmation à l'utilisateur qu'il s'agit bien de la bonne clé via son empreinte. Si l'utilisateur confirme que l'empreinte est bonne, ssh procèdera à son enregistrement afin de permettre sa vérification lors des visites suivantes.

Dès lors, il existe plusieurs manières de contrôler l'identité du serveur :

1) En s'assurant que l'empreinte de la clé présentée est identique à celle présente sur le serveur(ssh-keygen -l) ;
2) En ajoutant la clé manuellement au préalable dans le fichier known_hosts du poste client.

Comme vous pouvez le constater, ces méthodes sont assez lourdes à mettre en œuvre lorsque l'on administre plusieurs équipements à l'aide de ce protocole.

Le mécanisme SSHFP permet d'offrir plus de souplesse dans cette étape capitale de vérification.

Tout d'abord, assurez-vous manuellement que l'identité du serveur (l'empreinte de la clé présentée) lors de la première connexion est bien celle de la clé publique présente sur le serveur, comme le montre l'exemple ci-dessous :

```
etusio@srvssh:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub
```

```
Last login: Tue Dec  2 10:38:13 2025 from 192.168.56.11
etusio@srvssh:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub
256 SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWlM4 root@srvssh (ECDSA)
```

A comparer à l'empreinte affichée sur le client :

```
ssh etusio@srvssh.local.sio.fr
The authenticity of host ' srvssh.local.sio.fr (192.168.56.10)' can't be
established.
ECDSA key fingerprint is
SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWlM4.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'srvssh.local.sio.fr, 192.168.56.10' (ECDSA)
to the list of known hosts.
etusio@srvssh.local.sio.fr's password:
```

SSHFP est un mécanisme permettant de publier les empreintes de clés publiques du serveur SSH dans la zone DNS de l'entreprise.

Q9. Expliquer l'intérêt de cette démarche.

Le protocole SSHFP est utilisé afin de sécuriser la première authentification. Le seul inconvénient est de devoir comparer les clés à la main.
Dans notre cas, le client SSH interroge le DNS afin de comparer l'empreinte reçue du serveur avec celle stockée dans le DNS. Si elles correspondent, la connexion est validée automatiquement sans poser de question à l'utilisateur.